

# 1. Print name, DOB, and mobile number

- Write a C program to print your name, date of birth, and mobile number.
- Expected Output:
  - Name : Alexandra Abramov DOB : July 14, 1975  
Mobile : 99-9999999999

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // Print Name
6      printf("Name   : Alexandra Abramov\n");
7
8      // Print Date of Birth
9      printf("DOB    : July 14, 1975\n");
10
11     // Print Mobile Number
12     printf("Mobile : 99-9999999999\n");
13
14     // Indicate successful execution
15     return(0);
16 }
```

## 2. Compute series sum

$$S = 1 + 1/2 + 1/3 + \dots + 1/50$$

- Write a C program to calculate the value of S where  $S = 1 + 1/2 + 1/3 + \dots + 1/50$ .

Expected Output:

Value of S: 4.50

```
1  #include <stdio.h>
2  int main() {
3      float S = 0;
4      int i;
5
6      // Loop to calculate the sum
7      for(i=1; i<=50; i++) {
8          S += (float)1/i;
9      }
10
11     // Print the result
12     printf("Value of S: %.2f\n", S);
13
14     return 0;
15 }
```

### 3. Find all divisors of a given integer

- Write a C program that finds all the divisors of an integer.

Test Data:

Input an integer: 45

Expected Output:

All the divisor of 45 are:

1

3

5

9

15

45

$$45 = 1 \times 45$$

$$45 = 3 \times 15$$

$$45 = 5 \times 9$$

Divisor : 1, 3, 5, 9, 15, 45

```
1  #include <stdio.h>
2  int main() {
3      int x, i;
4
5      // Get an integer input from the user
6      printf("\nInput an integer: ");
7      scanf("%d", &x);
8
9      // Print all the divisors of x
10     printf("All the divisors of %d are: ", x);
11     for(i = 1; i <= x; i++) {
12         if((x%i) == 0){
13             printf("\n%d", i);
14             printf("\n");
15         }
16     }
17
18     return 0;
19 }
```

## 4. Print positions and values of elements in an array $< 5$

- Write a C program to read an array of length 5 and print the position and value of the array elements of value less than 5.

Test Data:

Input the 5 members of the array:

15

25

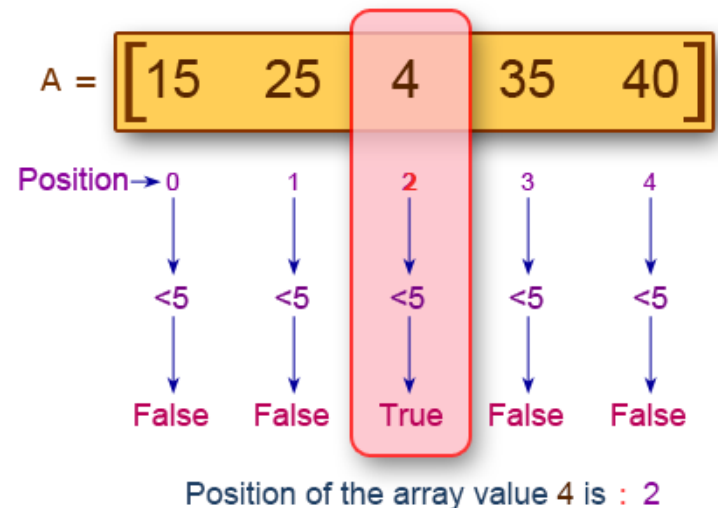
4

35

40

Expected Output:

$A[2] = 4.0$



```
1  #include <stdio.h>
2  #define AL 5 // Define the size of the array as 5
3  #define MAX 5 // Define a maximum value for comparison
4
5  int main() {
6      float N[AL]; // Declare an array of size 5 to hold floating-point numbers
7      int i;
8
9      // Prompt for user input
10     printf("Input the 5 members of the array:\n");
11
12     // Read and store user input into the array
13     for(i = 0; i < AL; i++) {
14         scanf("%f", &N[i]);
15     }
16
17     // Iterate through the array and print elements less than MAX
18     for(i = 0; i < AL; i++) {
19         if(N[i] < MAX) {
20             printf("A[%d] = %.1f\n", i, N[i]);
21         }
22     }
23
24     return 0;
25 }
```

# 5. Reverse array elements by swapping positions

- Write a C program to read an array of length 5, change the first element by the last, the second element by the fifth and the third element by the fourth. Print the elements of the modified array.

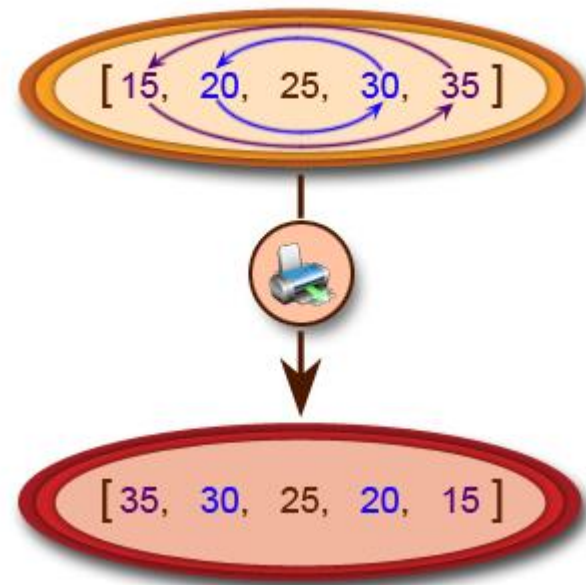
Test Data:

Input the 5 members of the array:

15  
20  
25  
30  
35

Expected Output:

array\_n[0] = 35  
array\_n[1] = 30  
array\_n[2] = 25  
array\_n[3] = 20  
array\_n[4] = 15



```
1  #include <stdio.h>
2  #define AL 5
3  int main() {
4      // Define the size of the array
5      int array_n[AL], i, temp;
6
7      // Prompt user to input 5 elements of the array
8      printf("Input the 5 members of the array:\n");
9      for(i = 0; i < AL; i++) {
10         scanf("%d", &array_n[i]);
11     }
12
13     // Swap the first half of the array with the second half
14     for(i = 0; i < AL; i++) {
15         if(i < (AL/2)) {
16             temp = array_n[i];
17             array_n[i] = array_n[AL-(i+1)];
18             array_n[AL-(i+1)] = temp;
19         }
20     }
21
22     // Print the modified array
23     for(i = 0; i < AL; i++) {
24         printf("array_n[%d] = %d\n", i, array_n[i]);
25     }
26
27     return 0;
28 }
```

## 6. Find the smallest element in an array and its position

- Write a C program to read an array of length 5 and find the smallest element and its position.
- Test Data:

Input the length of the array: 5

Input the array elements:

25

35

20

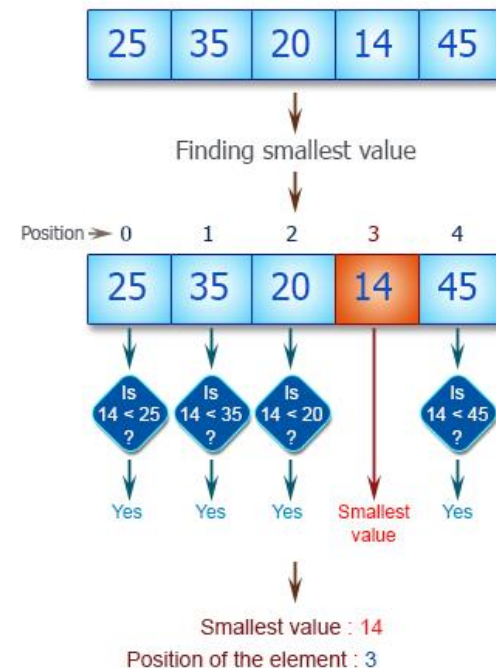
14

45

Expected Output:

Smallest Value: 14

Position of the element: 3



```
1  #include <stdio.h>
2  int main() {
3      int e, i, sval, position;
4
5      // Prompt user to input the length of the array
6      printf("\nInput the length of the array: ");
7      scanf("%d", &e);
8
9      // Declare an array of length 'e'
10     int v[e];
11
12     // Prompt user to input the array elements
13     printf("\nInput the array elements:\n ");
14     for(i = 0; i < e; i++) {
15         scanf("%d", &v[i]);
16     }
17
18     // Initialize 'sval' and 'position' with the first element of the array
19     sval = v[0];
20     position = 0;
21
22     // Find the smallest value and its position in the array
23     for(i = 0; i < e; i++) {
24         if(sval > v[i]) {
25             sval = v[i];
26             position = i;
27         }
28     }
29
30     // Print the smallest value and its position
31     printf("Smallest Value: %d\n", sval);
32     printf("Position of the element: %d\n", position);
33
34     return 0;
35 }
```

## 7. Print block 'F' and a large 'C'

- Write a C program to print a block F using the hash (#), where the F has a height of six characters and width of five and four characters. And also print a very large 'C'.
- Expected Output:

```
#####
#
#
#####
#
#
#

#####
##      ##
#
#
#
#
#
##      ##
#####
```

```

1  #include <stdio.h>
2
3  int main()
4  {
5      // Print a line of hashes
6      printf("#####\n");
7
8      // Print a single hash
9      printf("#\n");
10
11     // Print a single hash
12     printf("#\n");
13
14     // Print a line of hashes
15     printf("#####\n");
16
17     // Print a single hash
18     printf("#\n");
19
20     // Print a single hash
21     printf("#\n");
22
23     // Print a single hash
24     printf("#\n");
25
26     return(0);
27 }

```

```

1  #include <stdio.h>
2
3  int main()
4  {
5      // Print top line of pattern
6      printf("    #####\n");
7
8      // Print second line of pattern
9      printf("  ##      ##\n");
10
11     // Print lines 3 to 7 of pattern
12     printf(" #\n");
13     printf(" #\n");
14     printf(" #\n");
15     printf(" #\n");
16     printf(" #\n");
17
18     // Print bottom line of pattern
19     printf("  ##      ##\n");
20
21     // Print last line of pattern
22     printf("    #####\n");
23
24     return(0);
25 }

```